Predicting Machine Failures in Milling Machine: End-to-End Machine Learning Pipeline on the AI4I 2020 Dataset

Serra Ozkasap
Machine Learning and Data Mining, Digital Transformation Management
University of Bologna
serra.ozkasap@studio.unibo.it

***Abstract*—Unplanned machine stops interrupt production, create scrap and force repairs at the worst possible moment. In this project I build an end-to-end pipeline, following CRISP-DM, that predicts failures of a milling machine from five sensor readings in the AI4I 2020 Predictive Maintenance dataset. Since only 3.39% of the records are failures, I evaluate the models with recall, precision and the F2-score instead of accuracy, and I select every hyperparameter with stratified cross-validation on the training set. Two physically motivated features, mechanical power and the difference between process and air temperature, turn out to be the most important inputs. A single decision tree recovers the failure rules of the dataset from the sensor data alone, while distance-based and probabilistic models perform clearly worse because the failures occur at sharp thresholds and at both extremes of the operating range. Tree ensembles give the best results: the final random forest catches 57 of the 68 test failures with only 9 false alarms (F2 = 0.843) and is the most stable model over ten random splits. A cost matrix shows how the decision threshold should move with the cost of a missed failure, and nearly all remaining misses are tool wear failures, which are random by construction in this dataset.**

***Index Terms*—predictive maintenance, imbalanced classification, decision trees, random forest, gradient boosting, CRISP-DM**

I. Introduction

In manufacturing, a machine that stops without warning affects much more than the machine itself. The production flow is interrupted, the parts being processed at the moment of the failure often become scrap, and the repair has to be organised under time pressure. Predictive maintenance tries to avoid this by estimating, from the data the machine already produces, whether a failure is likely in the near future, so that the intervention can be planned at a convenient time. My background as an industrial engineer, and my thesis work on anomaly detection for a digital twin, are the reasons why I chose this problem for the course project.

I work on the AI4I 2020 Predictive Maintenance dataset [1], published on Kaggle and on the UCI repository. It is a synthetic dataset that reproduces the behaviour of a milling machine: each of its 10,000 records contains the product quality type (low, medium or high), five sensor readings (air temperature, process temperature, rotational speed, torque and tool wear), the binary target *Machine failure* and five flags describing the cause of the failure (tool wear failure, heat dissipation failure, power failure, overstrain failure and random failure). The dataset is small enough to be analysed in detail, but it contains the main difficulties of real maintenance data: failures are rare (339 records, 3.39%), some failure causes depend on combinations of sensors rather than on a single value, and part of the failures are random by definition.

The goal of the project is twofold: to predict whether a record corresponds to a failure using only information that is available before the failure happens, and to understand which operating conditions lead to it. I follow the six phases of CRISP-DM [2], and I compare several classifiers seen in the course, from a decision tree to distance-based and probabilistic models and tree ensembles, always under the same evaluation protocol.

This is an individual project: I carried out all phases, from business understanding to evaluation, as well as this report and the presentation. The analysis is implemented in Python with pandas, scikit-learn [3] and seaborn in a Google Colab notebook, and the full commit history, the notebook and the report are available in the project repository on GitHub.

II. Proposed Method

A. Business Understanding and Choice of Metric

The two possible errors of a failure predictor have very different costs. A missed failure (false negative) leads to an unplanned stop, scrap and possibly a damaged tool, while a false alarm (false positive) only causes an unnecessary inspection, that is, a short and planned stop. I therefore treat the recall of the failure class as the main metric. Because a model that raises an alarm on every record would reach a recall of one and still be useless, I also monitor precision, and I compare and tune the models with the F2-score, which combines the two but treats recall as twice as important as precision ($\beta = 2$) I also report the area under the precision-recall curve (PR-AUC), which does not depend on the decision threshold and measures how well the predicted probabilities rank failures above normal records; a random classifier scores the failure rate, 0.034. Accuracy is shown only as a reference: a model that always predicts "no failure" already reaches 96.61%.

B. Data Understanding

The data quality checks showed no missing values and no duplicated rows, and all values are physically plausible: there are no negative readings, the temperatures lie between 295 and 314 K, and the process temperature is higher than the air temperature in every record. Comparing the target with the five failure flags revealed 27 inconsistent records (0.27%): nine failures without any cause and eighteen records flagged as random failure without a recorded machine failure. Since the correct label cannot be known, I kept them and treat them as label noise.

The visual analysis showed that the sensors behave very differently in the two classes (Fig. 1). Failures happen at clearly higher torque (median 53.7 Nm against 39.9 Nm) and with more worn tools (165 against 107 minutes), while rotational speed shows failures at both very low and very high values. Two pairs of sensors are strongly correlated: air and process temperature ($r = 0.88$) and rotational speed and torque ($r = -0.88$). The failure rate also decreases with product quality, from 3.92% for type L to 2.09% for type H.

After preparing the data, I went back to Data Understanding to check whether the extreme values were errors and to measure non-linear feature relevance, following the iterative nature of CRISP-DM. With the interquartile range rule, 69 records are outliers in torque, and 89.9% of them are failures; an Isolation Forest fitted on the five sensors flagged 300 records, whose failure rate (17.3%) is almost six times that of the remaining records. The extreme values are therefore not measurement errors but the very conditions under which machines fail, and I did not remove or clip them. At the same time, these 300 records contain only 52 of the 339 failures, which shows that unsupervised anomaly detection alone is not enough and motivates the supervised approach.

C. Data Preparation

I removed the two identifiers, which carry no information about the machine, and the five failure flags, which are only known after the failure has happened and would therefore cause data leakage. The product type was encoded as an ordinal variable (L = 0, M = 1, H = 2), since its categories are ordered and the failure rate follows that order.

Two of the failure causes described in the dataset depend on combinations of sensors, so I derived two features with a physical meaning: the temperature difference $\Delta T = T_{\text{process}} - T_{\text{air}}$ and the mechanical power $P = \tau \cdot \omega$, where τ is the torque and ω the rotational speed converted to rad/s. Mutual information, i.e. the information gain of each feature with respect to the target, computed on the training set, confirmed the choice: torque (0.047), power (0.047) and rotational speed (0.035) are the most informative features, and ΔT is more informative than either temperature alone. Rotational speed is a useful example of why I did not rely on Pearson correlation alone: its correlation with the target is only -0.04, because failures occur at both ends of its range, while its mutual information is the third highest. All eight features were kept.

TABLE I
MEAN F2 IN FIVE-FOLD CROSS-VALIDATION ON THE TRAINING SET (CV F2) AND RESULTS ON THE TEST SET (68 FAILURES AMONG 2,000 RECORDS).

Model	CV F2	Recall	Precision	F2	PR-AUC
Baseline (always 0)	–	0.000	0.000	0.000	0.034
Decision tree (unlimited)	0.758	0.735	0.758	0.740	0.566
Decision tree (depth 10)	0.773	0.706	0.828	0.727	0.649
Decision tree (balanced)	0.753	0.824	0.577	0.759	0.776
KNN ($k = 71$, threshold 0.10)	0.695	0.735	0.400	0.630	0.579
Naive Bayes (threshold 0.20)	0.497	0.485	0.330	0.444	0.353
Random forest (tuned)	**0.807**	**0.838**	0.864	**0.843**	**0.871**
Gradient boosting (tuned)	0.800	0.824	0.889	0.836	0.832

The data were then split into a training set (8,000 records, 271 failures) and a test set (2,000 records, 68 failures), stratified on the target so that both sets keep the same failure rate. For the scale-sensitive models I standardised the features with a scaler fitted on the training set only and then applied to the test set.

D. Modeling and Evaluation Protocol

I trained the models one by one, from the simplest to the more complex, so that the weaknesses of each model could motivate the next one: a baseline that always predicts the majority class, a decision tree, k-nearest neighbours (KNN), Gaussian Naive Bayes, a random forest and gradient boosting. Every hyperparameter, class weight and decision threshold was selected with five-fold stratified cross-validation on the training set, using the mean F2-score, and the test set was used only once per model to report its final result. With only 68 failures in the test set, one failure more or less changes recall by about 1.5 percentage points; cross-validation averages over all 271 training failures and gives a more reliable basis for these choices. For KNN and Naive Bayes, which have no class weights, I tuned the decision threshold on cross-validated probabilities instead of using the default value of 0.5. Both tree ensembles were tuned with the same grid search: for the random forest the number of features considered at each split, the minimum leaf size and class weights recomputed in each bootstrap sample; for gradient boosting the number of boosting rounds, the learning rate and the tree depth. In the evaluation phase I compared the final models with ROC and precision-recall curves, repeated the train-test split ten times (random subsampling), chose the decision threshold with a cost matrix and drew a learning curve.

III. RESULTS

Table I summarises the cross-validated and test results of all models.

Decision tree: A tree grown without limits reaches a perfect score on the training set but an F2 of 0.740 on the test set, a clear sign of overfitting. The grid search selected a maximum depth of 10, which narrows the gap between training and test results; the best tree with balanced class weights catches 56 of the 68 test failures instead of 48, at the cost of 41 false alarms instead of 10. Which of the two is preferable depends on the real ratio between the cost of a missed failure and that of an

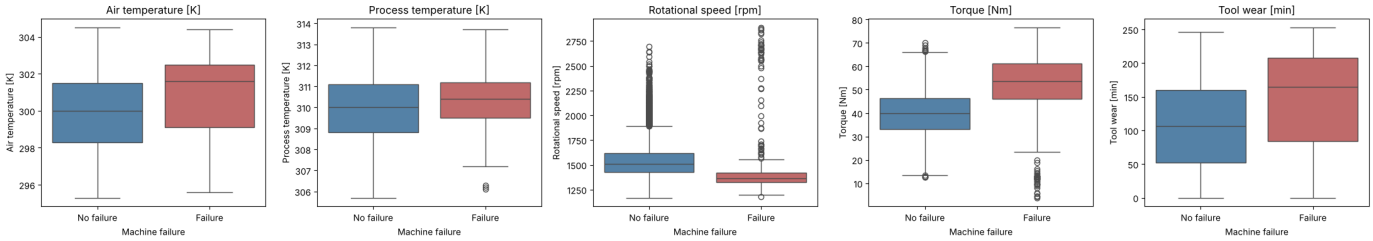


Fig. 1. Distribution of the five sensors for normal records and failures (Section 3 of the notebook). Failures concentrate at high torque and tool wear, and at both extremes of rotational speed.

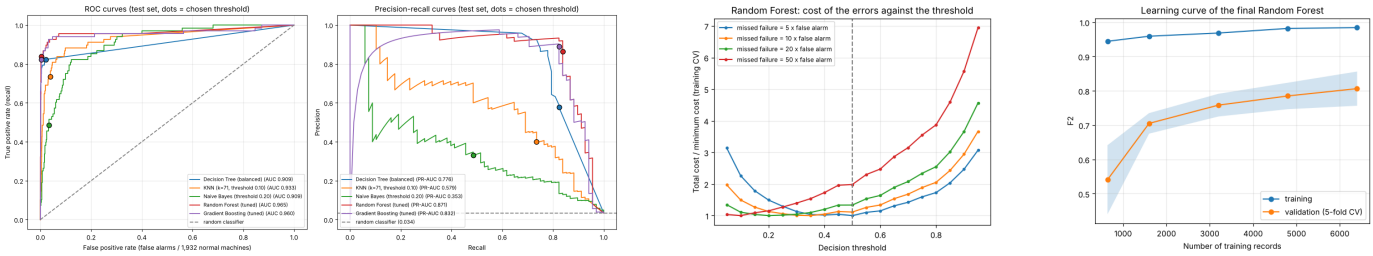


Fig. 2. Evaluation of the final models (Section 6 of the notebook). Left: ROC (a) and precision-recall (b) curves on the test set; the dots mark the threshold each model actually uses. Centre (c): total cost of the random forest's errors against the threshold for four cost ratios between a missed failure and a false alarm, on cross-validated training predictions. Right (d): learning curve of the random forest.

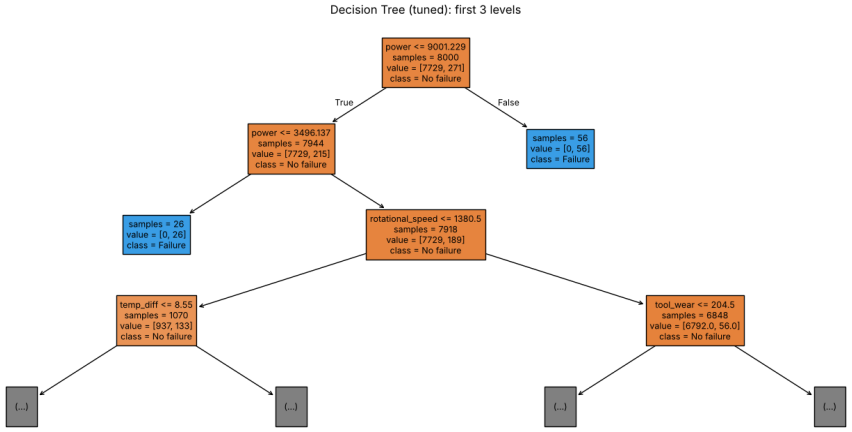


Fig. 3. First three levels of the tuned decision tree (depth 10). The splits reproduce the power failure (power below 3,496 W or above 9,001 W), heat dissipation failure (speed below 1,380 rpm and ΔT below 8.55 K) and tool wear failure rules.

inspection. The most interesting result is the structure of the tree (Fig. 3). Without ever seeing the failure flags, its first splits reproduce the failure definitions of the dataset: power below 3,496 W or above 9,001 W leads directly to two pure leaves of failures (power failure), rotational speed below 1,380 rpm combined with a temperature difference below 8.55 K isolates heat dissipation failures, and tool wear above 204.5 minutes opens the branch of tool wear failures. Power and ΔT, the two derived features, are also the two most important features of the model (importance 0.386 and 0.293).

KNN: Standardising the features raised the recall of KNN from 0.279 to 0.338 with $k = 5$, confirming the scaling decision. Cross-validation showed, however, that with the default majority vote the F2-score falls as k grows, because the neighbourhood of almost every record is dominated by normal machines. Lowering the decision threshold had a much larger effect than the choice of k: with $k = 71$ and a threshold of 0.10 the cross-validated F2 rose from 0.48 to 0.69. On the test set this model catches 50 failures but raises 75 false alarms, remaining below the tree because a distance cannot represent the sharp thresholds that define the failures, and because features with little information still contribute to it.

Naive Bayes: Naive Bayes is the weakest model. Forcing equal priors raises its recall from 0.294 to 0.824 but multiplies the false alarms from 30 to 278, and its PR-AUC stays at 0.353, since changing the prior only moves the decision threshold without changing how the records are ranked. The low quality of its probabilities comes from two violated assumptions: the features are far from independent (torque and power have a correlation of 0.98 within the normal class), and

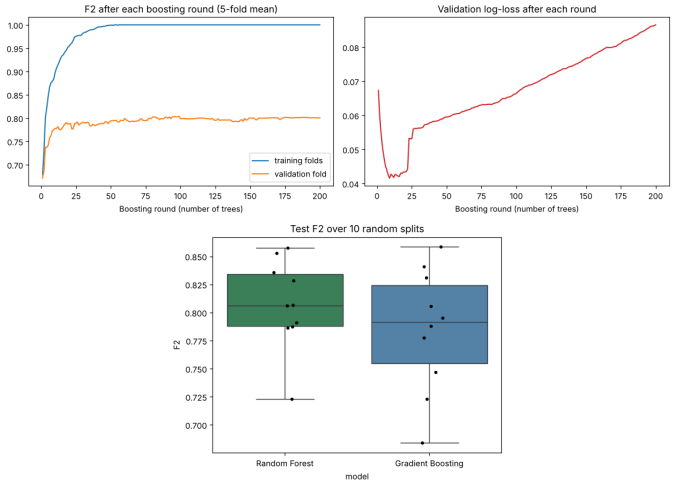


Fig. 4. (a) Gradient boosting round by round (5-fold mean): the training F2 reaches 1 after about 50 rounds while the validation F2 stays near 0.80, and the validation log-loss rises after the first rounds. (b) Random subsampling: test F2 of the two ensembles over ten random 80/20 splits.

the failures do not follow a single normal distribution, as power failures occur at both very low and very high power.

Random forest: A forest averages many trees, each grown on a bootstrap sample and choosing each split among a random subset of features, which reduces the variance of a single tree. With default settings it already ranks the records much better than any tree (PR-AUC 0.878), but at the 0.5 threshold it is cautious and catches only 51 failures. The out-of-bag estimate, computed on the records left out of each bootstrap sample, stopped improving at about 200 trees. Cross-validation selected class weights recomputed in each bootstrap sample, four candidate features per split and at least five records per leaf (CV F2 0.807, the highest of all models), and the default threshold remained the best. On the test set the forest catches 57 failures with 9 false alarms: one failure more than the balanced tree, with 32 fewer false alarms.

Gradient boosting: Boosting builds shallow trees in sequence, each fitted to the residual errors of the previous ones, and therefore reduces bias rather than variance. Its tuned version (200 rounds, learning rate 0.2, depth 5; CV F2 0.800) catches 56 failures with 7 false alarms, practically the same result as the forest. Fig. 4a shows the main difference between the two ensembles: adding trees to a forest never makes it overfit more, while adding boosting rounds does, since after about 50 rounds the model is perfect on the training folds and the new trees only memorise them, pushing the probabilities towards 0 or 1 (PR-AUC 0.904 for the default model, 0.832 for the tuned one).

Errors by failure mode: Using the failure flags only after training, to explain the errors, both ensembles catch all 13 power failures and almost all heat dissipation (29 and 28 of 29) and overstrain failures (16 and 14 of 16). Nine of the eleven failures missed by the forest are tool wear failures, and the other two have no recorded cause. In the data generation the tool is replaced or fails at a random time between 200 and 240 minutes of wear: 790 records fall in this window, but only 43 of them are tool wear failures, so no sensor value can tell which tool will fail. These errors are irreducible with the available features.

IV. Evaluation

The ROC and precision-recall curves (Fig. 2a,b) both place the two ensembles on top (ROC-AUC 0.965 and 0.960, PR-AUC 0.871 and 0.832). They disagree on the weaker models, which shows the limit of ROC on unbalanced data: since the false positive rate divides the false alarms by 1,932 normal machines, Naive Bayes still reaches a ROC-AUC of 0.909, the same as the balanced tree, while its PR-AUC of 0.353 reflects that it catches only 33 failures; this is why I reported PR-AUC for every model. Because the two ensembles differ by only one failure on the test set, I repeated the stratified split ten times (Fig. 4b): the random forest reaches a mean F2 of 0.807 ($\pm$0.040) against 0.785 ($\pm$0.054), is better in seven splits and has the higher recall, so I chose it as the final model. With a cost matrix (Fig. 2c), the default threshold of 0.5 is optimal when a missed failure costs five times an inspection, while for a cost ratio of 20 the best threshold drops to 0.2 and the forest catches 63 of the 68 test failures with 95 false alarms. The learning curve (Fig. 2d) is still rising, from an F2 of 0.54 with 640 training records to 0.81 with 6,400, which indicates that more failure records would further improve the model.

V. Conclusions

The results show that the most useful steps of the pipeline were the analysis of the data and the derivation of physically meaningful features. Models that learn thresholds directly from the data clearly outperform distance-based and probabilistic ones: a single decision tree already recovers the physical failure rules, and the final random forest reaches the best and most stable balance between missed failures and false alarms. The remaining misses are almost all tool wear failures, which are random by construction; in a real plant they would be handled by a planned tool replacement before 200 minutes rather than by a classifier. The decision threshold is ultimately a business choice: 0.5 is a good default, and a lower threshold is justified only if a missed failure costs at least an order of magnitude more than an inspection. The study also has clear limitations: the data are synthetic, random failures cannot be predicted by construction, 27 records have inconsistent labels, and the real costs of a missed failure and of an inspection are unknown, so the trade-off between recall and precision can only be discussed under assumptions.

References

[1] S. Matzka, “Explainable artificial intelligence for predictive maintenance applications,” in *Proc. 3rd Int. Conf. Artificial Intelligence for Industries (AI4I)*, 2020, pp. 69–74.

[2] R. Wirth and J. Hipp, “CRISP-DM: Towards a standard process model for data mining,” in *Proc. 4th Int. Conf. Practical Applications of Knowledge Discovery and Data Mining*, 2000, pp. 29–39.

[3] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.